

Seminar Thesis Report on Research on the Parallelization of the DBSCAN Clustering Algorithm for Spatial Data Mining Based on the Spark Platform

Mohammed Afaan Sajjad Hussain Shaikh (3844933)
Shaikhm@students.uni-marburg.de

Ram Binay Gupta (3848283)
Guptar@students.uni-marburg.de

Philipps-Universität Marburg
FB12 Mathematics and Computer Science Department

Abstract

In this work, we present an efficient parallel version of the DBSCAN clustering algorithm using Apache Spark. Our approach is inspired by the design proposed by Huang et al. [5] We implement the core ideas from their work and add our own optimizations to improve scalability and clustering accuracy on large spatial datasets. The dataset is first divided into a grid, with each cell processed independently. To ensure clusters are not broken at cell boundaries, we replicate points along the edges—these are called ghost points. For faster neighbor search, which is the most time-consuming part of DBSCAN, we use a KD-tree index inside each partition. Finally, clusters formed in different partitions are merged using a Union-Find strategy to maintain global consistency. Experiments show that our distributed approach significantly reduces runtime for large datasets while achieving clustering accuracy comparable to serial DBSCAN and Python’s `sklearn` implementation.

1 Introduction

Clustering helps find patterns in large spatial datasets. DBSCAN is a popular algorithm for this task—it can detect clusters of any shape and handles noise well. However, it becomes very slow on large datasets and is often too much for a single machine to handle. In this work, we implement a distributed version of DBSCAN using Apache Spark. We use grid partitioning to split the data, ghost points to preserve clusters across boundaries, KD-trees for faster neighbor searches, and Union-Find to merge clusters correctly. This approach improves scalability while keeping clustering accuracy high.

2 DBSCAN Terminology

Points are classified as:

- **Core:** At least $minPts$ neighbors within radius ϵ .
- **Border:** Lies within ϵ of a core point, but itself is not a core point.
- **Noise:** Does not belong to any cluster.
- **Ghost Point:** A replicated point used to merge local clusters in distributed DBSCAN.

3 Methodology

3.1 Grid Partitioning

Split the dataset into a grid of cells. Each cell is processed independently, which allows parallel execution and speeds things up. The cell size is calculated as: $= \epsilon \times cellFactor$. To do this, we look at the minimum and maximum coordinates in each dimension, figure out how many cells we need, give each cell a unique ID, and then assign every point to its correct cell based on where it’s located.

3.2 Ghost Point Replication

Points that lie near the edges of a cell are copied to neighboring cells. This buffer area is created by expanding the cell outward in all directions. The original paper suggested a buffer size of 0.1ϵ , but we also tested with 1ϵ . Copying these ghost points prevents clusters from breaking apart at cell boundaries. A larger buffer gives better clustering accuracy, but it also means more data gets copied around, which increases communication and computation overhead.

3.3 Local DBSCAN

Each partition runs the DBSCAN algorithm on its own set of points. To make neighbor searches faster, we build a KD-tree index for the points in that partition. For every point, we check how many neighbors lie within distance ϵ using the KD-tree. If a point has fewer than $minPts$ neighbors, it is marked as noise. Otherwise, it becomes a core point and starts forming a cluster by expanding to its neighbors. This local clustering step lets all partitions work in parallel before we merge the results globally (Algorithm 1).

3.4 Neighbour Search Using KD Tree

In the original approach, the range query checks every point against every other point in a partition. This brute-force method compares each point with all others, leading to a time complexity of $O(n^2)$. To make this faster, we replaced the naive range query with a KD-tree. A KD-tree organizes points in a spatial structure that helps skip irrelevant regions during searches.

Algorithm 1 Local DBSCAN with KD-tree Neighbor Search

```

1: Build KD-tree for points
2: clusterId  $\leftarrow$  0
3: for each point  $p$  in points do
4:   if  $p.visited$  then
5:     continue
6:   end if
7:    $p.visited \leftarrow true$ 
8:   neighbors  $\leftarrow$  KDTreeRangeQuery( $p, \epsilon$ )
9:   if  $|neighbors| < minPts$  then
10:     $p.clusterId \leftarrow -1$  {Noise}
11:   else
12:    clusterId  $\leftarrow$  clusterId + 1
13:     $p.clusterId \leftarrow$  clusterId
14:    expandCluster( $p$ , neighbors, clusterId,  $\epsilon$ , minPts)
15:   end if
16: end for

```

This brings the complexity down to roughly $O(n \log n)$, which makes a big difference for large datasets.

3.5 Cluster Merging

Clusters formed in different partitions might actually belong together. To fix this, we look at points that sit on partition boundaries. If a boundary point is a core point in two different partitions, those clusters get merged. We use Union-Find to handle this merging efficiently. It keeps track of which clusters are connected by linking their IDs together. But there’s a catch: collecting all these relationships and sending them to the driver node can slow things down and eat up memory. To get around this, we switched to GraphX. It treats cluster relationships as edges in a graph and runs connected components across worker nodes. This spreads the work around, keeps the driver from getting overloaded, and scales much better.

4 Experimental Setup

The experiments were carried out on a Mac machine running macOS 15.5 with 14 cores and 24 GB RAM. For the Spark configuration, we used 6 executors with 1 core each and allocated 3 GB of memory per executor. The datasets used include 50K, 100K, 5M, and 10M points in both 2D (Densired.2) and 3D (Densired.3) formats.

5 Performance Analysis

5.1 Runtime Comparison

For small datasets, parallel DBSCAN doesn’t help much because the overhead of distributing the work outweighs the benefits. But as the dataset grows, the parallel version starts to shine—it cuts down runtime significantly and scales much better Fig. 1. Speedup improves with dataset size, showing that parallel processing becomes more efficient as data increases. We also noticed that

the 2D dataset speeds up better than the 3D one. The extra dimension adds overhead, making parallelization a bit less effective in 3D (Fig. 2).

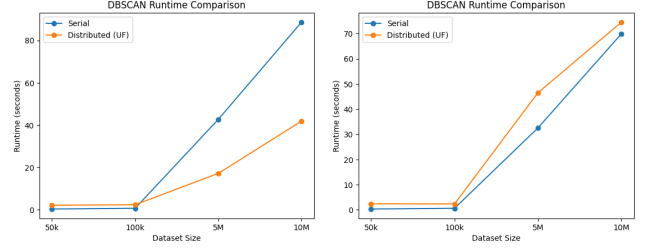


Figure 1: Runtime comparison between Serial and Parallel DBSCAN for densired.2 (left) and densired.3 (right) datasets. Parameters: $\epsilon = 0.03$, $b = 0.1$, $minPts = 50$

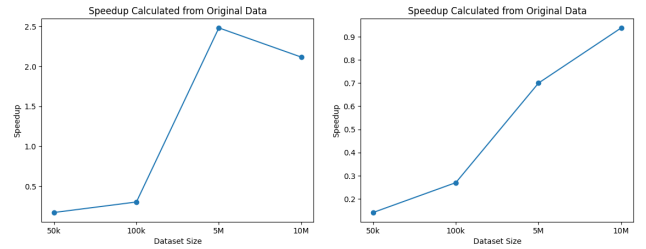


Figure 2: Speedup of Parallel DBSCAN over Serial DBSCAN for densired.2 (left) and densired.3 (right) datasets. Parameters: $\epsilon = 0.03$, $b = 0.1$, $minPts = 50$

5.2 Runtime Components

Several things contribute to the total execution time. Neighborhood search takes the biggest chunk, especially as the dataset grows. Without the KD-tree optimization, this part alone would eat up nearly 90% of the runtime. I/O operations from reading and writing files also add some cost, but they become less noticeable on larger datasets. The actual clustering work increases with the number of clusters and how dense the data is Fig. 3.

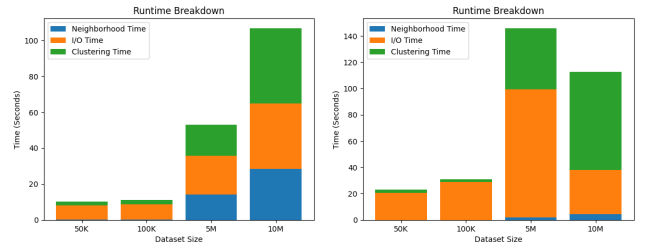


Figure 3: Runtime component breakdown for densired.2 (left) and densired.3 (right) datasets showing contributions of neighborhood search, I/O, and clustering computation.

5.3 Shuffle and Communication Overhead Analysis

As the dataset grows, more data gets shuffled between partitions, so shuffle volume jumps up a lot (Fig. 4).

Interestingly, shuffle read is always heavier than shuffle write, which means there’s extra communication happening behind the scenes. We also saw that the 3D dataset (densired.3) creates more shuffle traffic than the 2D one (densired.2), so adding dimensions comes with higher communication costs.

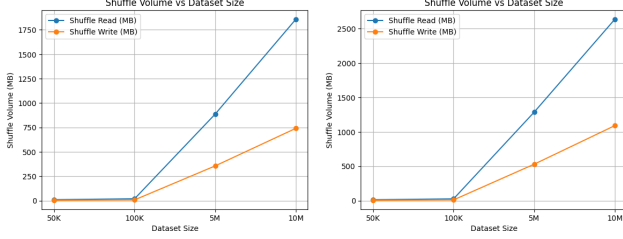


Figure 4: Shuffle read and write volume for densired.2 (left) and densired.3 (right) datasets with $\varepsilon = 0.03$, $b = 0.1$, and $\text{minPts} = 50$.

5.4 Ghost Point Replication and Buffer Impact

Increasing the buffer size leads to a lot more ghost point replication, especially when the dataset is large (Fig. 5). Points get copied across neighboring partitions multiple times, which drives up neighbor search costs and communication overhead. In some cases, this can even make the parallel version slower than running DBSCAN on a single machine (Fig. 6). A larger buffer does help catch clusters that stretch across boundaries, so accuracy improves. But that comes at a cost—more computation and data exchange mean longer runtimes.

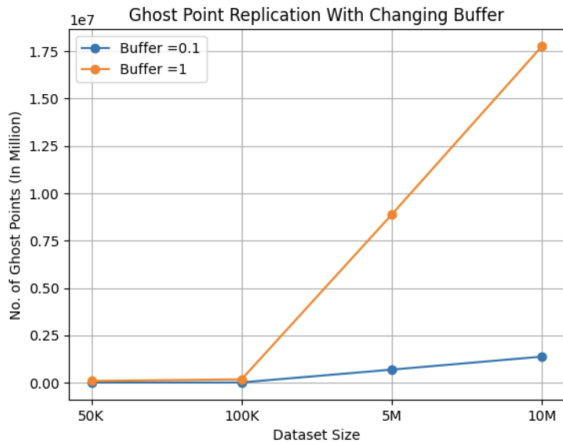


Figure 5: Ghost point replication for densired.2 dataset (50K, 100K, 5M, 10M) with $\varepsilon = 0.03$, buffer $b = 0.1$ and $b = 1$, and $\text{minPts} = 50$.

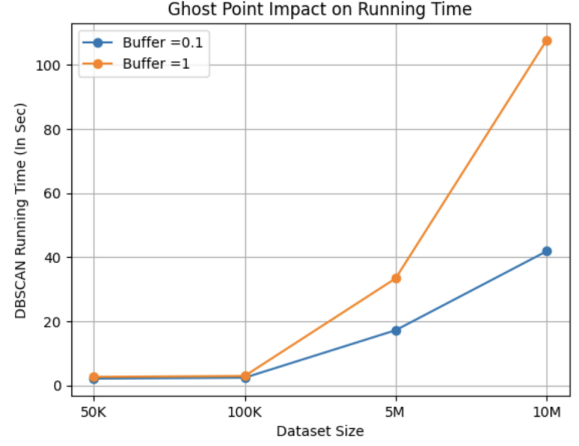


Figure 6: Impact of buffer size on DBSCAN runtime for densired.2 dataset (50K, 100K, 5M, 10M) with $\varepsilon = 0.03$, buffer $b = 0.1$ and $b = 1$, and $\text{minPts} = 50$.

5.5 Impact of Buffer Size on Clustering Accuracy

5.5.1 Impact of Buffer = 0.1

We compared the clustering results from our parallel implementation against both serial DBSCAN and Python’s `sklearn` DBSCAN (Fig. 7). With a buffer size of 0.1, the NMI score landed at 0.78 against both references, showing moderate agreement. Looking at cluster counts, serial DBSCAN found 2742 clusters, our parallel version found 2721, and `sklearn` found 2744.

Here’s what we found: distributed DBSCAN maintains decent accuracy even with a small buffer, but clusters do tend to break apart near partition boundaries. The parallel version also ends up with slightly fewer clusters than the other two—it misses a few around the edges. The takeaway is that a small buffer works okay, but you’ll see some fragmentation.

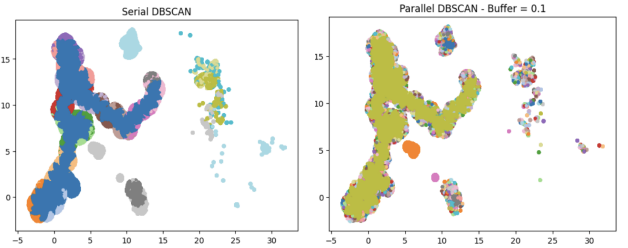


Figure 7: Clustering comparison for densired.2.5M dataset with $\varepsilon = 0.03$, $b = 0.1$, and $\text{minPts} = 50$. Left: Serial DBSCAN. Right: Parallel DBSCAN.

5.5.2 Impact of Buffer = 1

We also tested with a larger buffer size of 1 and compared the results against serial DBSCAN and `sklearn` (Fig. 8). This time, the NMI score jumped to 0.98 against `sklearn` and 0.99 against serial DBSCAN—almost perfect agreement. The cluster counts tell the same story: serial found 2742 clusters, our parallel version found 2743, and `sklearn` found 2744.

The findings are clear. Bumping up the buffer size makes a huge difference in clustering accuracy. Fragmentation at partition boundaries mostly disappears, and the parallel output ends up looking nearly identical to what serial DBSCAN and sklearn produce. So if you can afford the extra overhead, a larger buffer is definitely worth it.

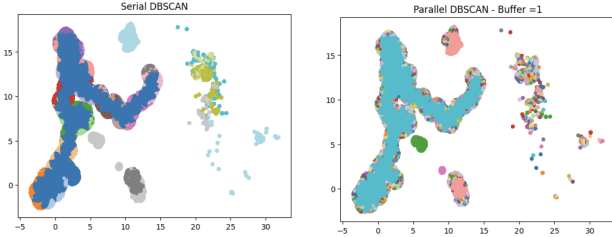


Figure 8: Clustering comparison for densired_2_5M dataset with $\varepsilon = 0.03$, $b = 1$, and $\text{minPts} = 50$. Left: Serial DBSCAN. Right: Parallel DBSCAN.

5.6 Impact of Epsilon (ε) and minPts on Runtime

Figure 9 shows how the DBSCAN runtime grows with increasing ε . As ε goes from 0.05 to 0.3, each point finds more neighbors, cluster expansion becomes heavier, and the overall execution time rises. At the same time, larger ε merges nearby points, so the total number of clusters gradually decreases.

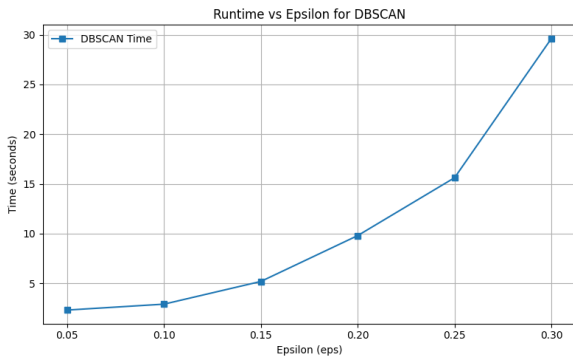


Figure 9: DBSCAN runtime vs epsilon (ε) for the densired_2_shrink dataset

Figure 10 illustrates the effect of raising minPts . With a higher threshold, fewer points qualify as core points, so fewer clusters are grown and less expansion work is performed. This reduces the overall runtime, especially for large datasets.

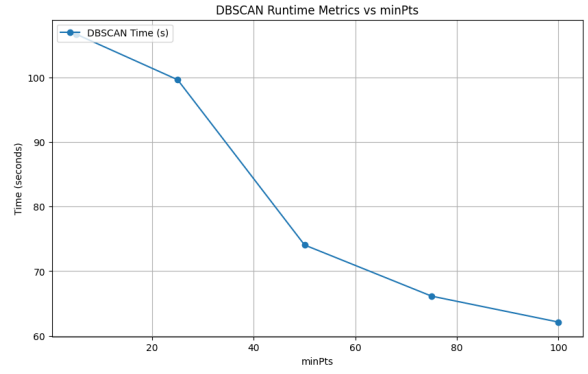


Figure 10: DBSCAN runtime vs minPts for the densired_2 dataset

5.7 Impact of Parallelism on Performance and Resource Usage

Figure 11 demonstrates how the number of worker nodes affects both runtime and memory/disk spill. With a single worker, the executor is overloaded: memory spills (4574MB) and disk spills (615MB) occur, prolonging runtime. Distributing the workload across 2–4 workers eliminates spill entirely and reduces execution time. However, adding too many workers can introduce shuffle overhead, occasionally making 3 workers slower than 2.

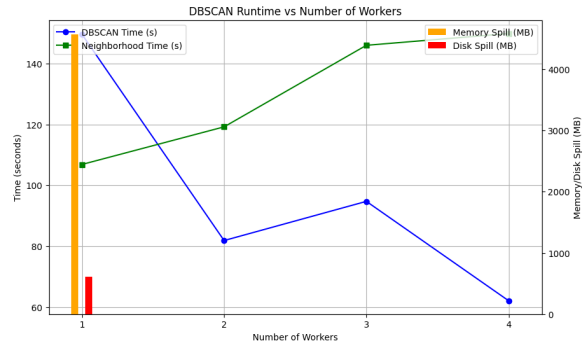


Figure 11: Effect of the number of workers on DBSCAN runtime and memory/disk spill (dataset: densired_2)

6 Findings and Observations

When the dataset is small, running DBSCAN in parallel doesn't really pay off. The overhead from setting everything up just gets in the way. But once the data gets larger, things start to look better—speedup improves, and the whole thing scales nicely. The biggest time sink by far is the neighborhood search. As datasets grow, that part just keeps taking longer and longer. That's why swapping in a KD-tree was so important. Without it, the runtime would be unbearable. Another thing we noticed is that shuffle volume shoots up as data increases, and that adds a fair amount of communication overhead. On the buffer side, there's a trade-off. A larger buffer means more ghost points get replicated, which drives up runtime. But it also does a much better job at keeping clusters connected across partition boundaries. So if you want cleaner, more accurate clus-

ters, a bigger buffer is the way to go—just be ready for the extra cost.

7 Missing Gaps in the Original Paper

Looking back at the original paper, we noticed a few things that weren't fully addressed. First, the merging strategy across distributed partitions wasn't clearly laid out. Without a solid merging approach, keeping clusters consistent across the whole dataset becomes tricky. Another gap was around optimization. The paper didn't go deep into techniques for speeding up neighborhood searches, which is a big deal when you're working with large datasets. That part really affects scalability and performance. We also found limited discussion on how the algorithm performs with high-dimensional data. The paper didn't really analyze how adding dimensions impacts runtime or clustering accuracy. Finally, cluster fragmentation at partition boundaries was mentioned as an issue, but there wasn't much done to actually fix it. That can hurt accuracy in a distributed setup.

8 Challenges and Issues Encountered

Along the way, we ran into a few challenges. The KD-tree helped speed up neighbor searches, but building and maintaining it came with its own computational cost. Merging clusters across different partitions also turned out to be tricky—we had to add extra logic to keep everything consistent globally. Making sure the clustering was accurate meant constantly checking our results against serial DBSCAN and sklearn using NMI scores. That took some back and forth. Another issue we saw was cluster fragmentation at partition boundaries, especially when we kept the buffer size small. And finally, all the data moving between partitions led to more shuffle operations, which slowed things down more than we'd have liked.

9 Optimization Techniques

We tried a few things to make the whole system run smoother. First, we switched to Kryo serialization instead of the default Java one. That cut down the size of data being shuffled between nodes, which sped things up quite a bit. We also used broadcast variables to share read-only data across workers without sending the same stuff over and over again—that saved some extra network traffic. The KD-tree was probably the biggest win. Since neighbor search is the slowest part of DBSCAN, speeding that up made a huge difference. It cut down runtime and helped things scale better with larger datasets. To make cluster merging cleaner, we brought in Union-Find. It helped us connect fragmented clusters across partitions without messing up accuracy. Lastly, we spent some time tuning Spark's resource settings—things like memory and core counts. Getting

those parameters right helped balance the workload and kept bottlenecks from forming. All these optimizations together made the whole system more efficient and scalable.

10 Conclusion

The distributed DBSCAN implementation improves clustering scalability and runtime performance for large spatial datasets while maintaining high clustering accuracy. Optimization of neighbor search and cluster merging is critical for distributed performance.

Acknowledgment

We sincerely thank Florian Siepe for his continuous support and guidance. We also thank Professor Thorsten Papenbrock for providing us the opportunity to work on this seminar.

Code Repository

The implementation of this project is available at: <https://github.com/Guptaram001/dbscan.git>

References

- [1] Apache Software Foundation. Rdd programming guide. <https://spark.apache.org/docs/latest/rdd-programming-guide.html>, 2026. Accessed: Dec. 11, 2025.
- [2] cp-algorithms contributors. Disjoint set union. https://cp-algorithms.com/data_structures/disjoint_set_union.html, 2025. Accessed: Jan. 11, 2026.
- [3] Desmos Studio PBC. Plot points. <https://www.desmos.com/calculator/mhq4hsncnh>, 2025. Accessed: Dec. 20, 2025.
- [4] GeeksforGeeks. Search and insertion in k dimensional tree. <https://www.geeksforgeeks.org/dsa/search-and-insertion-in-k-dimensional-tree/>, July 2025. Accessed: Dec. 21, 2025.
- [5] F. Huang, Q. Zhu, J. Zhou, J. Tao, X. Zhou, D. Jin, X. Tan, and L. Wang. Research on the parallelization of the dbscan clustering algorithm for spatial data mining based on the spark platform. *Remote Sensing*, 9(12):1301, 2017.
- [6] OpenAI. Chatgpt (large language model). <https://chat.openai.com>, 2023. Accessed: Feb. 10, 2026.